SECTION 1 • PROJECT INTRODUCTION

FoodHouse — Enterprise Food Ordering & Delivery Platform

Core Project Definition

"FoodHouse is a Java-based enterprise food ordering web application that explore restaurants, browse curated menus, manage an interactive real-tim place food orders with dynamic live delivery tracking through a modern w

30

Top Bangalore Restaurants

360

Curated Dishes Catalog

14

Jakarta Servlets (MVC)

SECTION 2 • TECHNOLOGY STACK

Production Technology Stack & Architectural Tooling

CLIENT VIEW

Frontend Tier

- ✓ HTML5 & Semantic Markup
- ✓ CSS3 (Glassmorphism & Responsive)
- ✓ Vanilla JavaScript (DOM & Fetch API)

BUSINESS LOGIC

Backend Tier

- ✓ Java SE 21 (LTS Release)
- ✓ Jakarta Servlet 6.0 Specification
- ✓ MVC Architecture Implementation

PERSISTENCE

Database Tier

- ✓ MySQL 8.0 Relational Engine
- ✓ JDBC (Java Database Connectivity)
- ✓ PreparedStatement Batch Insertion

SECTION 3 • PROJECT STRUCTURE

Eclipse Dynamic Web Project & Layered Package Organization

FoodHouse — Eclipse Dynamic Web Project Structure

Complete Java EE Layered Architecture (Model-View-Controller + Data Access Object Pattern)

Project Explorer: FoodHouse

- src/main/java
  - com.tap.controller (14 Servlets)
    - RestaurantController.java
    - MenuController.java
    - CartController.java
    - CheckoutController.java
    - OrderController.java
    - OrderHistoryController.java
    - LoginController.java & RegisterController.java
    - OrderStatusController.java & DeliveryAdminController.java
  - com.tap.model (7 POJO Entities)
    - User.java, Restaurant.java, Menu.java
    - Cart.java, CartItem.java, Order.java, OrderItem.java
  - com.tap.DAO & DAOImpl (5 Interfaces & Impls)
    - UserDAO.java / UserDAOImpl.java
    - RestaurantDAO.java / RestaurantDAOImpl.java
    - MenuDAO.java / MenuDAOImpl.java
    - OrderDAO.java / OrderDAOImpl.java
    - OrderItemDAO.java / OrderItemDAOImpl.java
  - com.tap.util (Utilities & Engines)
    - DBConnection.java, OrderLifecycleManager.java
    - EncryptExistingUsers.java, DBTableInit.java
- src/main/webapp
  - Restaurant.jsp (Home Catalog Portal)
  - Menu.jsp, Cart.jsp, Checkout.jsp
  - Login.jsp (Unified Auth), OrderHistory.jsp
  - Profile.jsp, DeliveryDashboard.jsp, AdminRestaurant.jsp

Controller Layer (Jakarta Servlets) — 14 Controllers

Handles client requests, route orchestration, session security, query parameter validation, and forwards dynamically to JSP presentation views via

`RestaurantController` `MenuController` `CartController` `OrderController` `OrderHistoryController` `OrderStatusController`

DAO & DAO Implementation Layer — Decoupled Persistence

Strict separation of business logic and database access using Java Interfaces (UserDAO, OrderDAO) and robust JDBC implementations utilizing Pre

`UserDAOImpl` `RestaurantDAOImpl` `MenuDAOImpl` `OrderDAOImpl` `OrderItemDAOImpl`

View Layer & Assets (JSP + CSS + JS) — Dynamic Rendering

Dynamic server-side views using Expression Language (EL) & JSTL, custom glassmorphism styling, zero-jump client-side cart updates, and mobile

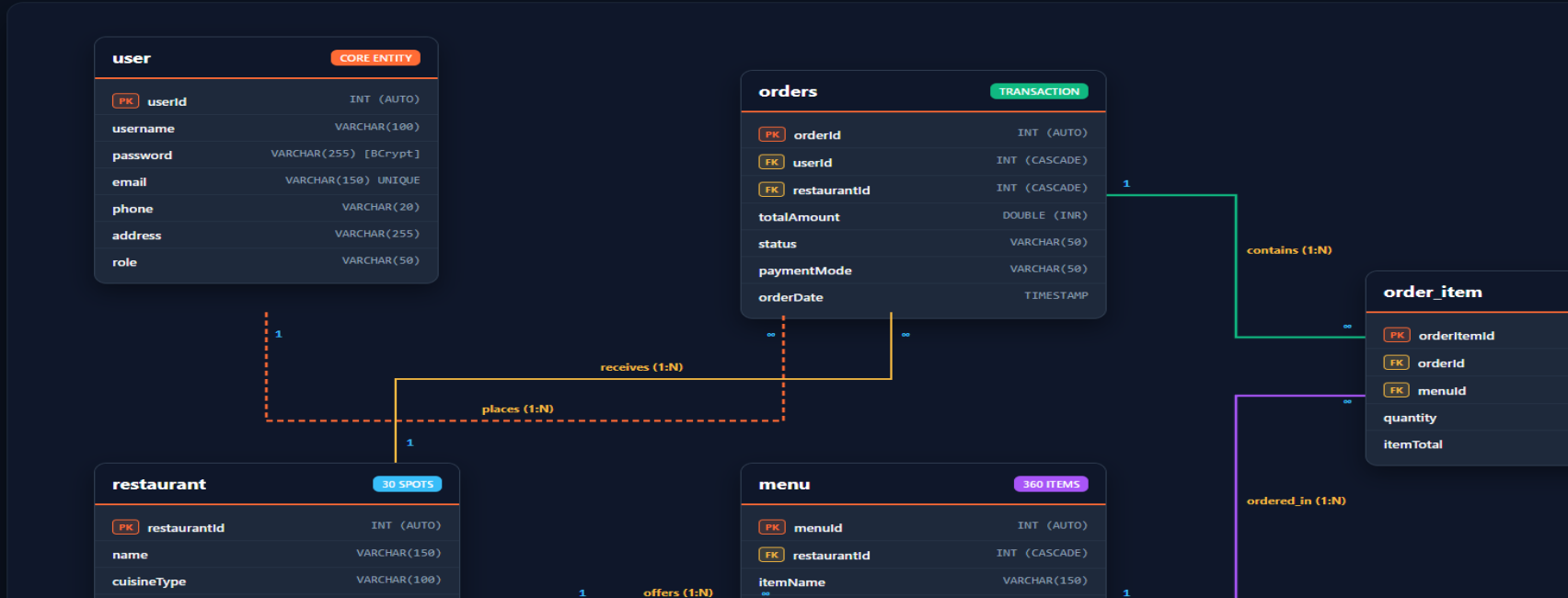
`Restaurant.jsp` `Menu.jsp` `Cart.jsp` `Checkout.jsp` `OrderHistory.jsp` `Login.jsp`

SECTION 4 • ER DIAGRAM

MySQL Normalized Relational Database Schema & Entity Relati

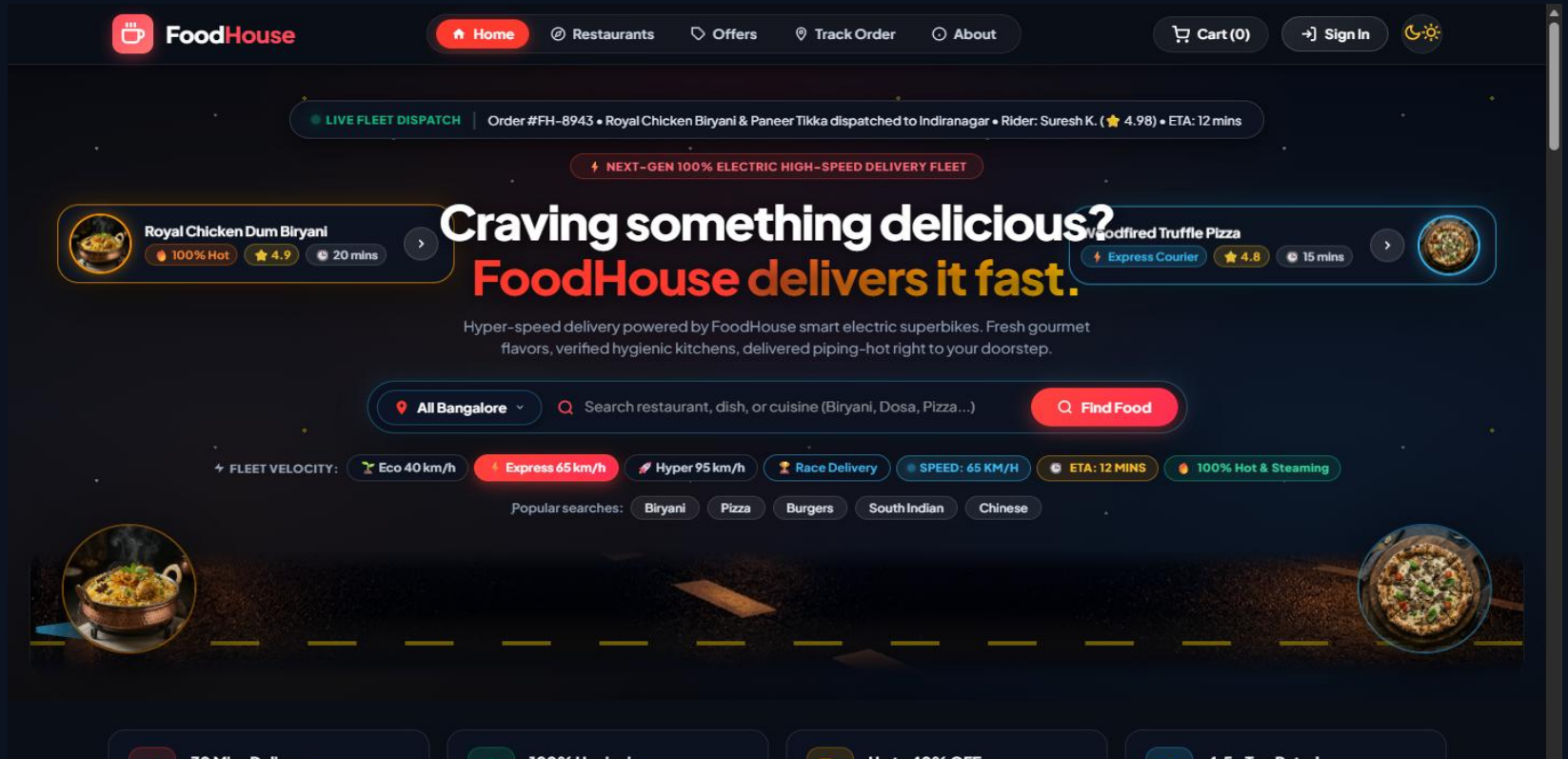
FoodHouse — MySQL Relational Database Schema

Complete 5-Table Entity-Relationship Diagram with Keys, Constraints & Cardinalities



SECTION 5 • PAGE SCREENSHOTS (1/6)

Home & Restaurant Discovery Portal (Restaurant.jsp)



Ke

• D
ret

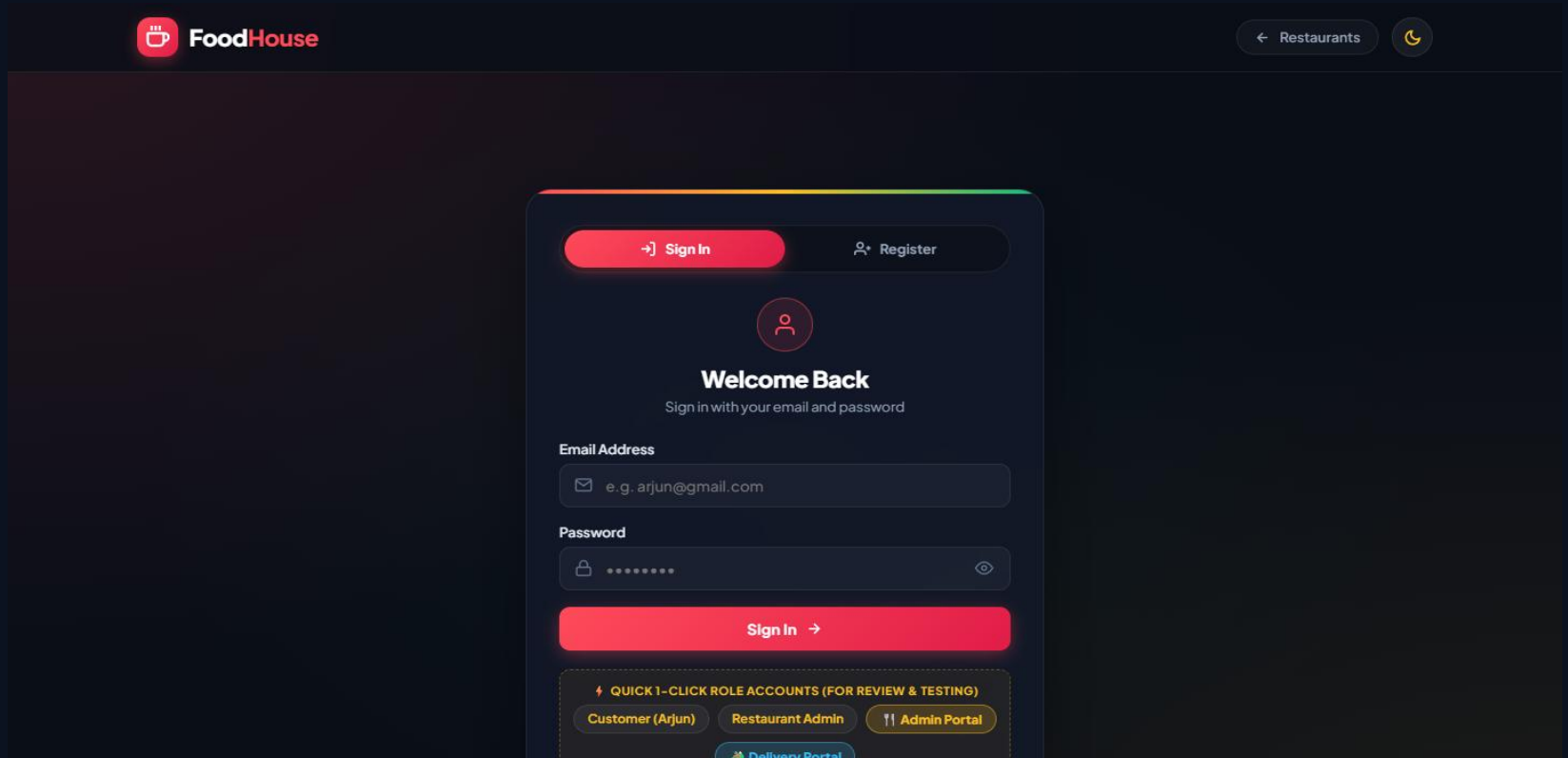
• L
Ko
Ba

• In
cui

• Li
Hy

SECTION 5 • PAGE SCREENSHOTS (2/6)

Unified Authentication Portal (Login.jsp & Register.jsp)



Se

• U
tab

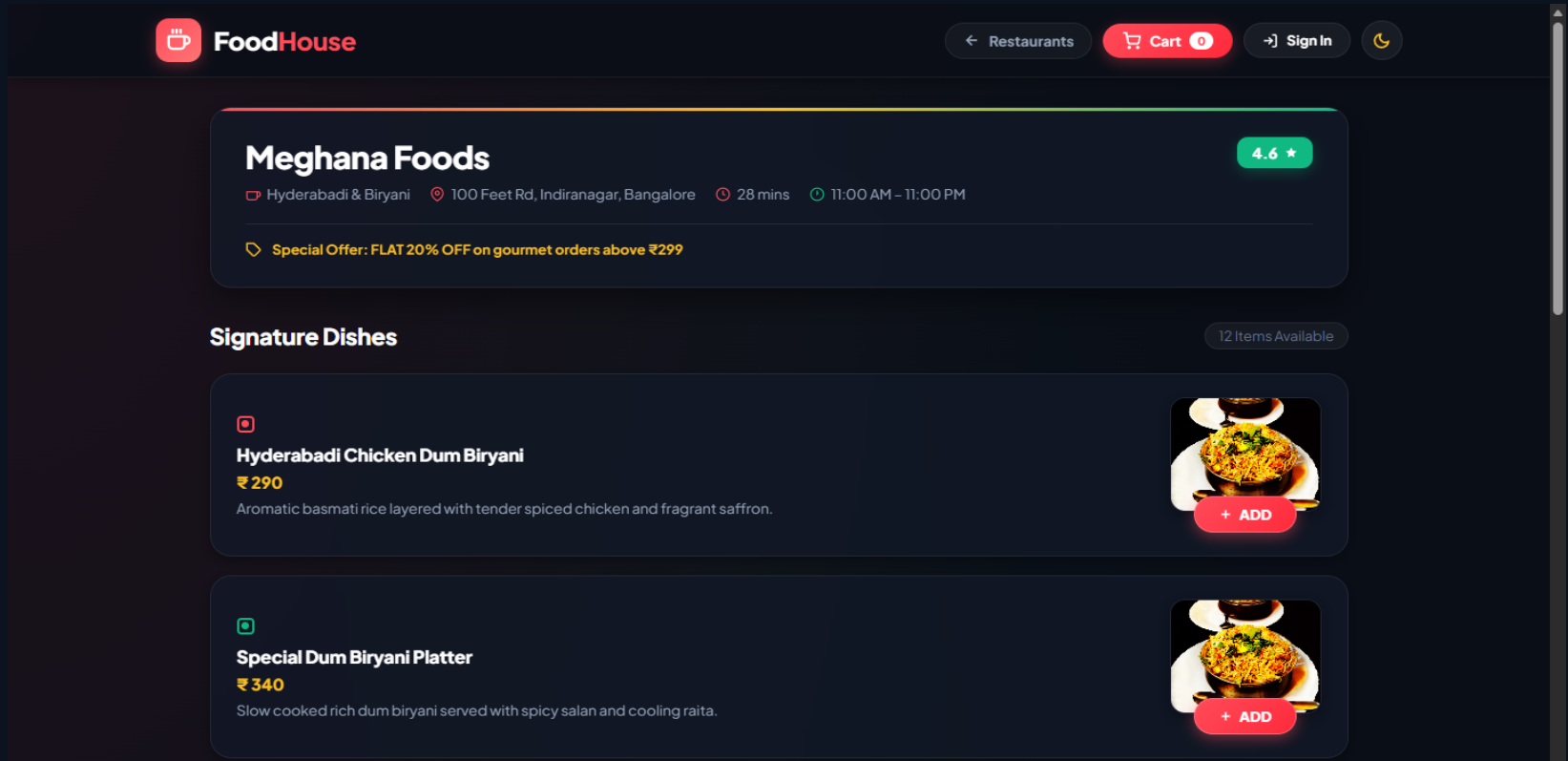
• E
has

• P
JDE

• H
pre
ide

SECTION 5 • PAGE SCREENSHOTS (3/6)

Curated Restaurant Menu & Dish Selection (Menu.jsp)



M

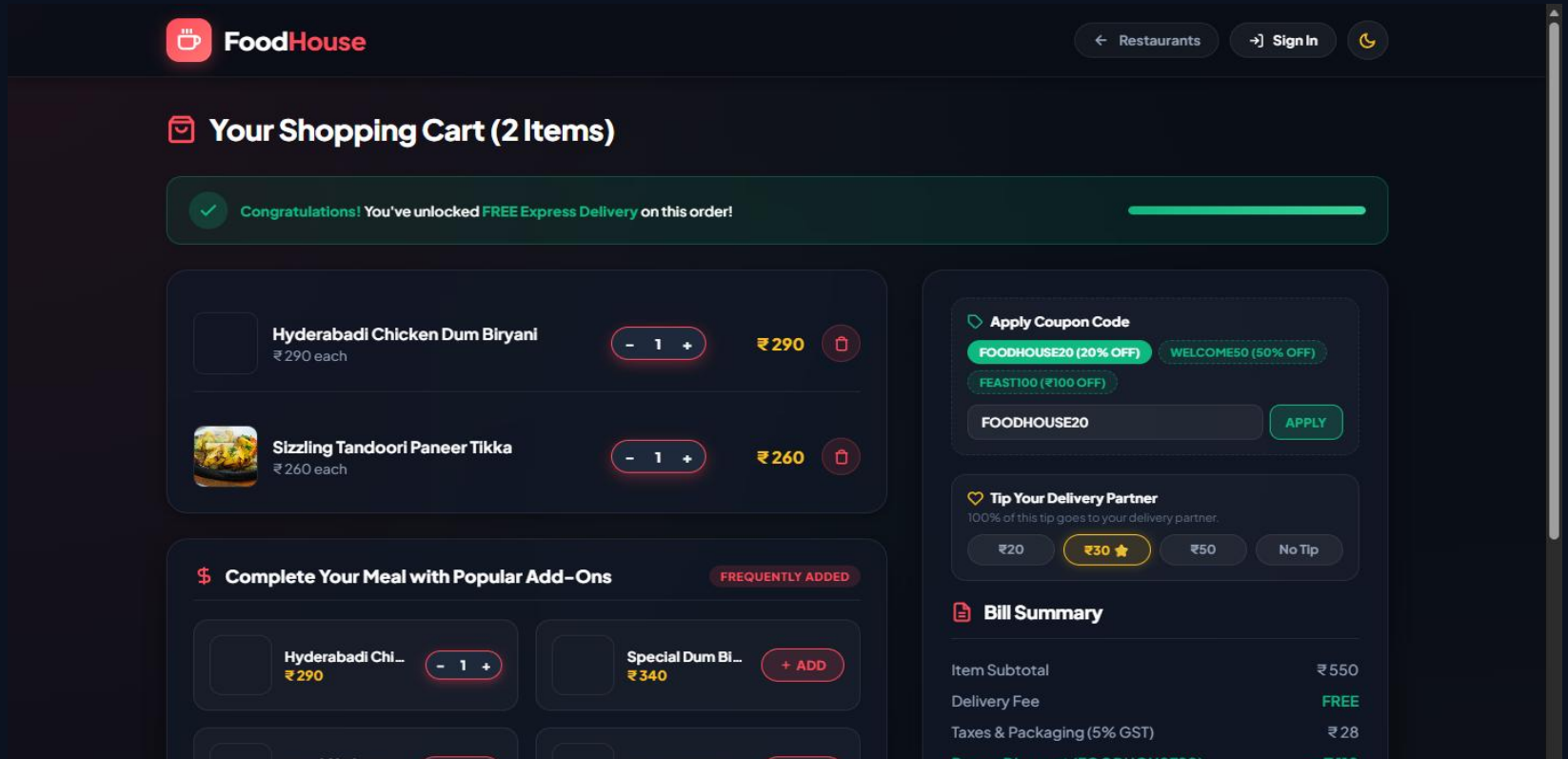
• R
qu
Me

• V
ind

• Z
pre
dis

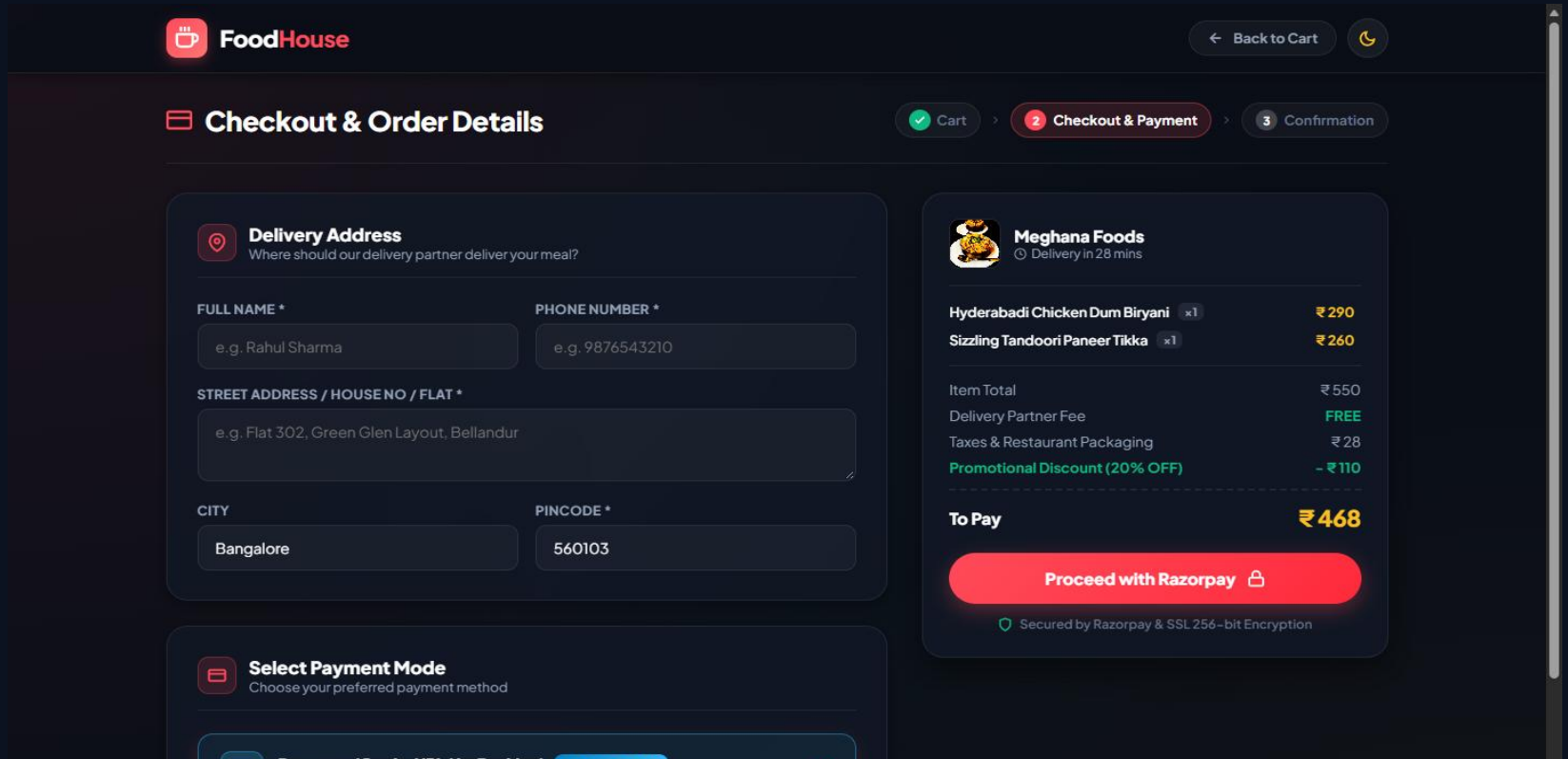
• In
dir

Interactive Shopping Cart & Financial Calculations (Cart.jsp)



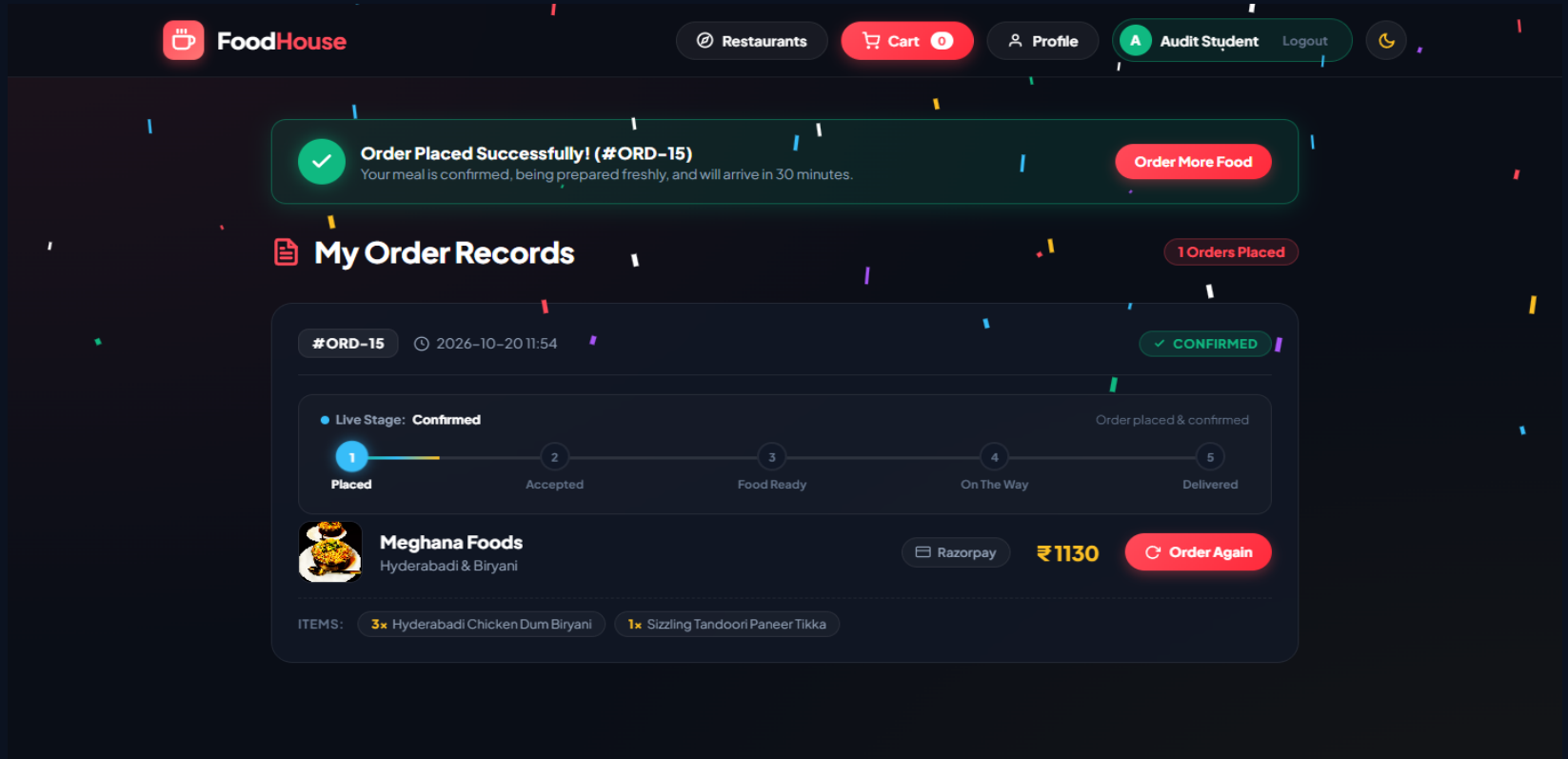
SECTION 5 • PAGE SCREENSHOTS (5/6)

Checkout & Payment Mode Selection (Checkout.jsp)



SECTION 5 • PAGE SCREENSHOTS (6/6)

Order History & Real-Time Tracking (OrderHistory.jsp)



Li

• R
pas

• D
(Co
→ C

• P
Or
Ma

• It
ord
ID.

SECTION 6 • NEW FEATURES ADDED

Key Architectural Innovations Beyond Base TAP Project Require

BACKEND ALGORITHM

Order Lifecycle State Machine

What it does: Built OrderLifecycleManager.java to manage deterministic, forward-only status progression (Confirmed → Delivered).

Why it matters: Eliminates static status values and models real-world kitchen preparation and fleet delivery tracking.

SECURITY UPGRADE

BCry Hard

What it does: Replaced plain-text passwo jBCrypt hashing and automated migratio

Why it matters: Guarantees enterprise pr breach vulnerabilities and protects user c

FRONTEND INNOVATION

Zero-Jump Dynamic Menu UX

What it does: Engineered asynchronous cart modification with scroll-anchor preservation on dish cards.

UI POLISH

Dyna Engi

What it does: Implemented dynamic Dar CSS custom properties and persistent sta

SECTION 7 • IMPORTANT ALGORITHMS

OrderLifecycleManager Finite State Machine & Transactional Pe

FoodHouse — Core Business Logic & State Machine Algorithm

OrderLifecycleManager Finite State Machine (FSM) & PreparedStatement Transactional Persistence

com.tap.util.Order

OrderLifecycleManager.java (State Progression Engine) • Production + Demo Dual Modes

```java
public class OrderLifecycleManager {
    // Real-world Timeline (mins) vs Demo Timeline (secs)
    public static final long DEFAULT_MINUTES_ACCEPTED = 2;
    public static final long DEFAULT_MINUTES_FOOD_READY = 5;
    public static final long DEFAULT_MINUTES_ON_THE_WAY = 10;
    public static final long DEFAULT_MINUTES_DELIVERED = 20;

    public static String computeDynamicStatus(Order order) {
        String current = order.getStatus();
        // Terminal States are protected from rollback
        if (isTerminalStatus(current)) return current;

        long elapsedMillis = System.currentTimeMillis() - order.getOrderDate().getTime();
        long elapsedMinutes = elapsedMillis / (60 * 1000);

        String next = current;
        if (elapsedMinutes >= DEFAULT_MINUTES_DELIVERED) next = "Delivered";
        else if (elapsedMinutes >= DEFAULT_MINUTES_ON_THE_WAY) next = "On The Way";
        else if (elapsedMinutes >= DEFAULT_MINUTES_FOOD_READY) next = "Food Ready";
        else if (elapsedMinutes >= DEFAULT_MINUTES_ACCEPTED) next = "Accepted";

        // Enforce Monotonic Forward Progression
        if (getStageNumber(next) > getStageNumber(current)) {
            orderDAO.updateOrderStatus(order.getOrderId(), next);
            return next;
        }
        return current;
    }
}
```

01 Stage 1: Order Placement & DB Ingestion
OrderController verifies user session and initiates transactional
Status: "Confirmed" • Time: T = 0 mins • PreparedStatem

02 Stage 2: Restaurant Kitchen Acceptance
Kitchen dashboard acknowledges incoming ticket; state update
Status: "Accepted" • Threshold: T ≥ 2 mins (or 15s in f

03 Stage 3: Culinary Prep & Quality Packaging
Food is prepared, sealed in hygienic tamper-proof package, and
Status: "Food Ready" • Threshold: T ≥ 5 mins (or 30s in

04 Stage 4: Fleet Dispatch & Live Delivery
Rider departs from restaurant; client UI renders pulsing bike ra
Status: "On The Way" → "Delivered" • Monotonic Non-Regr

SECTION 8 • MAJOR CHALLENGE FACED

Viewport Scroll Jump Disorientation & Password Encryption Mi

FoodHouse — Major Technical Challenge & Resolution

Evidence-Based Root Cause Investigation & Architectural Solution

1. The Problem

During user testing on large menus (e.g. **Meghana Foods** with 12 items), clicking **"+ ADD"** on bottom items triggered a standard HTTP POST form submission.

SYMPTOM The browser refreshed the page and violently **jumped the viewport scroll back to the very top (scrollY = 0)**, forcing the user to repeatedly scroll down to add multiple dishes.

Additionally, legacy database seed accounts stored plain-text passwords, creating a critical security vulnerability and credential mismatch.

2. Root Cause Analysis

Deep-dive into Menu.jsp and CartController.java revealed:

TECHNICAL CAUSE Standard JSP <form action="cart" method="post"> without AJAX interception caused an unavoidable document unload/reload sequence.

Because the browser default scroll restoration reset scroll position to 0 upon reload, client contextual position was lost entirely.

In the database layer, users seeded before security hardening lacked the BCrypt salt prefix ($2a$), causing BCrypt.checkpw() exceptions.

3. The Engineerin

ZERO-JUMP IMPLEMENTATION In
with smooth scroll anchor preser
cart badge update.

AUTOMATED DB MIGRATION Bui
records and migrate them autom

Verification: 100% verified via au
confirming **ScrollY difference** < 2

SECTION 9 • WHAT I'LL ADD NEXT

Future Architectural Enhancements & Production Roadmap

01

Live Payment Gateway

Razorpay / Stripe API

Integrate real online payments supporting UPI intent, credit/debit cards, net banking, and automatic webhook

02

Live GPS Fleet Tracking

Mapbox / Google Maps API

Render interactive map visualizer showing real-time delivery rider coordinates, route polylines, and dynamic

03

SMS & WhatsApp Alerts

Twilio Cloud Webhooks

Automated dispatch notifications, OTP verification at delivery handover, and live order confirmation alerts to